



DEPARTAMENTO DE INGENIERÍA INFORMÁTICA Y CIENCIAS DE LA COMPUTACIÓN
UNIVERSIDAD DE CONCEPCIÓN

Boletín 2: Árboles de Búsqueda

Evaluación teórica y experimental de Árboles AVL, Rojo-Negro y Splay

Estudiante:
Francisco Arentsen

Matrícula:
[Tu Número de Matrícula]

Profesor: José Fuentes
Ayudante: Leonardo Lovera

Asignatura:
Estructuras de Datos y Algoritmos
Avanzados (2026-2)

29 de septiembre de 2026

1. Resumen

En el presente boletín se realiza un estudio comparativo teórico y experimental de tres estructuras de datos de árboles de búsqueda binaria auto-balanceados: Árbol AVL, Árbol Rojo-Negro y Árbol Splay. La motivación principal es analizar sus desempeños en escenarios reales bajo patrones de acceso uniformes y no uniformes, permitiendo identificar la pertinencia de uso de cada estructura según los requisitos de frecuencia de acceso y restricciones de tiempo.

2. Análisis Teórico

Las estructuras analizadas ofrecen garantías teóricas sobre la complejidad temporal de las operaciones principales (`search`, `insert`, `delete`):

Cuadro 1: Comparativa de complejidad temporal y espacial.

Estructura	Búsqueda	Inserción	Eliminación	Espacio
Árbol AVL	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Rojo-Negro (<code>std::set</code>)	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Árbol Splay	$\mathcal{O}(\log n)$ amort.	$\mathcal{O}(\log n)$ amort.	$\mathcal{O}(\log n)$ amort.	$\mathcal{O}(n)$

Detalles del comportamiento teórico:

- **Árbol AVL:** Mantiene un balance estricto mediante la diferencia de alturas (factor de balance $\in \{-1, 0, 1\}$), favoreciendo búsquedas más rápidas en promedio.
- **Árbol Rojo-Negro:** Mantiene el balance mediante coloreado de nodos. Requiere menos rotaciones durante modificaciones, siendo altamente eficiente en inserciones/eliminaciones masivas.
- **Árbol Splay:** Reorganiza dinámicamente los nodos accedidos moviéndolos a la raíz mediante la operación *splay*. Destaca por su propiedad de localidad de referencia.

3. Implementaciones

- **Árbol Rojo-Negro:** Implementado utilizando el contenedor genérico `std::set` de la librería estándar de C++ (STL).
- **Árbol AVL y Splay:** Basados en implementaciones adaptadas a clases C++ genéricas (`template`), extendidas con métodos explícitos para `search`, `insert` y `delete`.

4. Resultados Experimentales

4.1. Entorno de Pruebas y Hardware

Los experimentos fueron ejecutados bajo un entorno controlado con las siguientes características técnicas:

- **Procesador:** [Ej: AMD Ryzen 7 5800X / Intel Core i7-11800H @ 3.2 GHz]
- **Memoria RAM:** [Ej: 16 GB DDR4 3200 MHz]
- **Almacenamiento:** [Ej: SSD NVMe M.2 512 GB]
- **Compilador y SO:** GCC 11.2, C++17, [Ubuntu 22.04 LTS / Windows 11]

4.2. Diseño Experimental

1. **Poblado del árbol:** Se insertaron n claves en cada estructura con $n \in \{10^3, 10^4, 10^5, 10^6\}$.
2. **Distribución de Consultas:** Se evaluaron consultas siguiendo distribuciones de probabilidad discretas no uniformes (ej: Geométrica / Poisson) para simular el sesgo de acceso exigido en la evaluación del Splay Tree.
3. **Medición de Tiempos:** Uso de la librería `<chrono>` de C++ para medir la ejecución promedio por consulta.

4.3. Resultados Obtenidos

Cuadro 2: Tiempo promedio de búsqueda (μs) según la frecuencia de acceso.

Estructura	Claves Frecuentes	Claves Poco Frecuentes	Consultas Repetidas
Árbol AVL	—	—	—
Rojo-Negro	—	—	—
Árbol Splay	—	—	—

5. Conclusiones

- **Comparativa Cuantitativa:** [Describir los resultados. Ej: *En escenarios con sesgo de acceso, el Árbol Splay resultó ser X veces más rápido en consultas repetidas que el Árbol AVL y Rojo-Negro*].
- **Toma de Decisiones y Uso Recomendado:**
 - Usar **Árbol Splay** cuando el patrón de acceso tenga alta localidad de referencia o consultas muy sesgadas.
 - Usar **Árbol Rojo-Negro / AVL** cuando se requieran garantías de respuesta estrictas e independientes del orden o frecuencia de las consultas.

6. Referencias

1. GeeksforGeeks. *Splay Tree Practice & Implementation*. <https://www.geeksforgeeks.org/>
2. Documentación de la C++ Standard Template Library (STL) para `std::set`.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.